

HELIOS: AI-Powered Autonomous Security Engineer

Product Tagline: Build with AI. Secure with HELIOS.

Comprehensive Engineering Specification & System Documentation

HELIOS is a next-generation autonomous cybersecurity application designed to solve security vulnerabilities commonly introduced into AI-generated web applications (e.g. Cursor, Bolt, Lovable, v0, Replit).

Unlike traditional static linters or superficial mockups, HELIOS delivers a real-world security tracker that probes live servers over HTTP, extracts actual security headers, detects critical misconfigurations, and verifies fixes via automated live re-testing.

1. Executive Summary

- HELIOS doesn't just detect vulnerabilities: It understands, investigates, proves, prioritizes, fixes, and verifies.
- 100% Real Evidence: Every risk score (0-100) is dynamically calculated from actual HTTP response headers.
- Live Re-testing: Engineers can click 'Re-Test' or command by voice to verify whether a patch resolved the issue.

2. Core Promise & Zero-Fake-Data Architecture

- Automatic Voice Greeting: Aloud greeting ('Welcome sir, how may I help you?') on site load and mic activation.
- Hands-Free Continuous Dialogue: Automatically re-arms listening after speaking answers.
- Smart Silence Detection: 1.5s silence timer prevents premature audio cutoffs.
- Intent Routing: Supports scanning, retesting, theme toggling, CVE explanations, and CISO exports.

3. Two-Way AI Voice Copilot (Gemini 3.5 + ElevenLabs)

4. Threat Modeling & Attack-Path Engine

- Identifies chained exploit paths (e.g., missing HSTS leading to SSL stripping and session hijacking).
- Maps blast radius, affected crown jewels, and prioritized remediation code snippets.

Technical Architecture & API Specifications

- POST /api/scan-url : Initiates real HTTP probe on target URL, audits security headers, and returns findings
- POST /api/retest-url : Re-probes live target to verify if a specific vulnerability ID is resolved or persists.
- POST /api/scan-code : Static analysis engine detecting SQLi, prompt injection, and hardcoded credentials
- POST /api/v1/voice/command : Natural language voice controller powered by Gemini 3.5 & rule heuristics
- GET /api/v1/voice/tts : High-fidelity English voice synthesis MP3 stream (ElevenLabs / Google fallback)

5. REST API Specifications

- Backend: Python 3.13 Standard Library (http.server, urllib, json, re) — Zero required third-party wheels
- Frontend: React 18, Tailwind CSS, Lucide Icons, Glassmorphism design system (Dark & Light modes).
- Audio & Speech: Web Speech API, HTML5 AudioContext, Web Audio synthesizers for acoustic chimes.
- AI Brain: Google Gemini 3.5 / 2.0 Flash with automatic fallback cascade.
- Voice Engine: ElevenLabs Voice API (Rachel model) with high-definition fallback.

6. Technology Stack

1. Clone repository: `git clone https://github.com/TeamName/KH001-TeamName.git`
2. Enter directory: `cd KH001-TeamName/src`
3. Launch autonomous server: `python server.py`
4. Open in browser: `http://localhost:3000/helios.html`

7. Quickstart & Verification Instructions

5. Click the Voice Orb or anywhere on the page to hear: 'Welcome sir, how may I help you?'